

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.:T039	Name:Bhavesh Ramakant Salaskar
Class:TYIT	Batch:1
Date of Assignment:14-07-2026	Date/Time of Submission:14-07-2026

Part A - Theory :

C# Delegate

What is a Delegate?

A **Delegate** is a special type in C# that stores the reference (address) of a method. It allows a method to be called indirectly through a delegate object. A delegate can call any method that has the **same return type and parameters** as the delegate declaration.

Why are Delegates Used?

- To call methods indirectly.
- To increase code flexibility.
- To implement callbacks.
- Used in event handling and GUI programming.

Real-Life Example

Think of a **TV Remote**.

- TV = Method
- Remote = Delegate

Instead of pressing buttons on the TV, we use the remote. Similarly, instead of calling a method directly, we use a delegate.

Syntax of Delegate

```
delegate returnType DelegateName(parameter_list);
```

Example

```
delegate void Operation(int a, int b);
```

Steps to Use a Delegate

Step 1: Declare the delegate

```
delegate void Operation(int a, int b);
```

Step 2: Create methods having the same signature

```
public void Add(int a, int b)  
{
```

```
    Console.WriteLine(a + b);  
}
```

Step 3: Create a delegate object

Operation obj;

Step 4: Assign a method to the delegate

obj = Add;

Step 5: Invoke the delegate

obj(10, 20);

or

obj.Invoke(10, 20);

Part B - Program

- a) Create methods add(), multiply(), subtract(), divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

Step 1: Start.

Step 2: Define a delegate Operation that accepts two integer parameters.

Step 3: Create a class calculate with methods for Addition, Multiplication, Subtraction, and Division.

Step 4: Create an object of the calculate class.

Step 5: Declare a delegate variable obj.

Step 6: Assign the delegate to the Add() method and call it with two integers.

Step 7: Assign the delegate to the Mul() method and call it with two integers.

Step 8: Assign the delegate to the Sub() method and call it with two integers.

Step 9: Assign the delegate to the Div() method and call it with two integers.

Step 10: Display the results of all arithmetic operations.

Step 11: Display "Bhavesh T039".

Step 12: Stop.

Code:

```
using System;  
delegate void Operation(int a, int b);  
class calculate  
{  
    public void Add(int a ,int b)  
    {  
        Console.WriteLine("Addition =" + (a + b));  
    }  
    public void Mul(int a, int b)  
    {  
        Console.WriteLine("Multiplication =" + (a * b));  
    }  
}
```

```

    public void Sub(int a, int b)
    {
        Console.WriteLine("Subtraction =" + (a - b));
    }
    public void Div(int a, int b)
    {
        Console.WriteLine("Division =" + (a / b));
    }
}
public class class1
{
    static void Main()
    {
        calculate c = new calculate();
        Operation obj;

        obj = c.Add;
        obj(10,20);

        obj = c.Mul;
        obj(4, 8);

        obj = c.Sub;
        obj(5, 8);

        obj = c.Div;
        obj(25, 5);
        Console.WriteLine("Bhavesh T039");
    }
}

```

Output:

```

Addition =30
Multiplication =32
Subtraction =-3
Division =5
Bhavesh T039

```

b) Write a program using multicast delegate.

Algorithm:

Step 1: Start.

Step 2: Define a delegate Message with no parameters.

Step 3: Create a class Mesg with methods Wel(), Hello(), and Th() to display messages.

Step 4: Create an object of the Mesg class.

Step 5: Declare a delegate variable obj.

Step 6: Assign the Wel() method to the delegate.

Step 7: Add the Hello() and Th() methods to the delegate using the += operator.

Step 8: Invoke the delegate to execute all three methods in sequence.

Step 9: Display "Bhavesh T039".

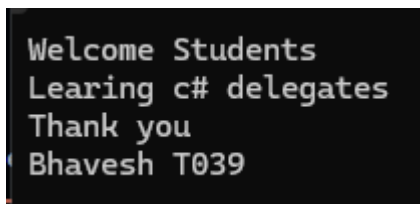
Step 10: Stop.

Code:

```
using System;
delegate void Message();
class Mesg
{
    public void Wel()
    {
        Console.WriteLine("Welcome Students");
    }
    public void Hello()
    {
        Console.WriteLine("Learning c# delegates");
    }
    public void Th()
    {
        Console.WriteLine("Thank you");
    }
}
public class Program
{
    static void Main(string[] args)
    {
        Mesg m = new Mesg();
        Message obj;

        obj = m.Wel;
        obj += m.Hello;
        obj += m.Th;
        obj();
        Console.WriteLine("Bhavesh T039");
    }
}
```

Output:

A screenshot of a terminal window with a black background and white text. The output consists of four lines: "Welcome Students", "Learning c# delegates", "Thank you", and "Bhavesh T039".

```
Welcome Students
Learning c# delegates
Thank you
Bhavesh T039
```

c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

Step 1: Start.

Step 2: Create a class Bankacc with properties Accnum and Balance.

Step 3: Use a private variable balance to store the account balance.

Step 4: Validate the Balance property so that only non-negative values are accepted.

Step 5: Create an object of the Bankacc class.

Step 6: Assign an account number and a valid balance.

Step 7: Display the account number and balance.

Step 8: Assign a negative balance value.

Step 9: Display an error message if the balance is negative.

Step 10: Stop.

Code:

```
using System;
class Bankacc
{
    public int Accnum { get; set; }
    private double balance;
    public double Balance
    {
        get { return balance; }
        set { if (value >= 0)
            balance = value;
            else
                Console.WriteLine("Balance cannot be negative");
        }
    }
}

class Program
{
    static void Main()
    {
        Bankacc b = new Bankacc();
        b.Accnum = 100;
        b.Balance = 5000;
        Console.WriteLine("Account number:"+b.Accnum);
        Console.WriteLine("Balance: "+b.Balance);

        b.Balance = -1000;
    }
}
```

Output:

```
Account number:100
Balance: 5000
Balance cannot be negative
```

Part C – Curiosity Question

1. A college wants to **calculate student performance**.

- a. Create methods to
- b. Calculate Total Marks
- c. Calculate Percentage
- d. Display Grade
- e. Display Pass/Fail

Note: Call each method using a delegate.

2. A college wants to automate the admission process.
When the user clicks **Confirm Admission**, several operations must happen automatically. Design a delegate-based solution.

Students decide:

- a. Verify Documents
- b. Generate Roll Number
- c. Allocate Division
- d. Send SMS

Note: For above operation just print a statement.